# High-Energy Particle Storms Simulation

## Comparative Analysis of Distributed (MPI/OpenMP) vs. GPU (CUDA)

Jacopo Mazzatorta

April 5, 2026

## 1 INTRODUCTION

The technological development has brought to light many new possibilities and applications in the scientific world.

Real-world phenomena that have always been out of reach in terms of costs and resources are now accessible thanks to High-Performance Computing (HPC) implementations. A relevant example is a high-energy particle storm. Through computational physics, the scientific community can now study radiation and its interactions with the physical world safely.

The goal of this project is to analyze and parallelize a simplified algorithm that simulates the effects of high-energy particle storms on a one-dimensional layer.

As I am not a physicist, the report focuses on the mathematical aspects and the architectural choices of my code implementations, rather than the theoretical physics behind them.

In particular, I present two parallel versions:

1. A hybrid **MPI + OpenMP** version targeting a multi-node CPU cluster.
2. A **GPU-accelerated** version using CUDA.

For both versions, I analyze the performance using hardware profilers (`perf stat` for CPU, `nsys` for GPU) to connect the observed speedups to the underlying architecture. Finally, I compare the results to evaluate when each approach is most effective.

## 2 TEST ENVIRONMENT & METHODOLOGY

Before analyzing the results, it is important to define the testing conditions. To ensure a fair comparison between the baseline and the parallel implementations, I followed a strict methodology.

**Table 1** Test cases used throughout the report.

| Name | Layer | Particles | Waves | Used for |
|---|---|---|---|---|
| test_02 | 30 K | 20 K | 6 | Strong, Weak, CUDA |
| personal_01 | 60 K | 20 K | 6 | Weak prog. 1 |
| personal_02 | 120 K | 20 K | 6 | Strong, Weak, CUDA |
| test_07 | 1 M | 5 K | 4 | Strong, Weak, CUDA |
| personal_03 | 2 M | 5 K | 4 | Weak prog. 2, CUDA |
| personal_04 | 4 M | 5 K | 4 | Weak prog. 2, CUDA |

### 2.1 Compilation Flags

All versions of the code were compiled with the highest level of optimization enabled.

- **CPU (GCC):** `-O3` combined with `-flto` (Link-Time Optimization) and architecture-specific tuning (`-march=znver1` for the AMD EPYC Naples nodes). I also enabled aggressive math optimizations (`-ffinite-math-only`, `-fno-signed-zeros`) to allow the compiler to vectorize operations that do not require IEEE compliance.
- **GPU (NVCC):** `-O3` with `-Xptxas -O3 -Xcompiler -O3` to target the compute capability of the available GPU.

### 2.2 Test Cases

I defined six problem sizes covering a wide range of computational cases. Each test specifies a layer size and the number of particles within the storm.

**Table 2** Baseline execution times (optimized code, 1 process, 1 thread). Median of 6 runs.

| Test Case | $T_s$ (s) | Range (s) |
|---|---|---|
| 30 K | 1.157 | [1.157, 1.159] |
| 60 K | 2.293 | [2.289, 2.314] |
| 120 K | 4.621 | [4.613, 4.638] |
| 1 M | 7.080 | [7.036, 7.238] |
| 2 M | 26.499 | [26.436, 26.573] |
| 4 M | 58.080 | [57.952, 58.125] |

### 2.3 Baseline

Measuring speedup requires a fair baseline. Using the original unoptimized sequential code would inflate the results, since the parallel code includes significant algorithmic optimizations (lookup tables, memory alignment, SIMD hints) that have nothing to do with parallelism.

Instead, I used the optimized parallel code compiled with all optimizations enabled, but executed with a single MPI process and a single OpenMP thread. This shows the speedup that only comes from parallelism and hardware exploitation.

**Code optimizations alone**

On the 1 M test, the original sequential code takes 278.8 s while the optimized single-process baseline takes 7.08 s, which is a 39× improvement before any parallelism is applied.

### 2.4 Measurement Protocol

Each configuration was executed at least 6 times. I report the median execution time and the range (minimum to maximum) as error bars in charts. The median is more robust than the mean against outliers caused by cluster noise.

### 2.5 Numerical Validation

All three versions of the code (baseline, MPI+OpenMP, and CUDA) produce bit-identical results for every test case. The output includes the position and energy value of the maximum after each storm wave, and these match exactly in all configurations and problem sizes. This confirms that the parallelization does not introduce numerical errors.

## 3 DISTRIBUTED MEMORY PARALLELIZATION (MPI + OPENMP)

The first parallel implementation uses a hybrid approach, combining MPI (Message Passing Interface) with OpenMP. MPI handles communication mostly between different nodes, while OpenMP only exploits the cores within each node.

### 3.1 Host Architecture & Setup

For the experiments, I used the High-Performance Computing cluster of Università La Sapienza. Each computing node is a dual-socket machine equipped with AMD EPYC Naples processors:

- **Sockets:** 2 per node.
- **Cores:** 16 cores per socket, 32 total per node.
- **Cache:** L1 32 KB/core, L2 512 KB/core, L3 8 MB per CCX (shared by 2 cores). Total L3 per node: 64 MB.
- **NUMA:** 4 domains per node. Memory is not equally close to all cores.

I exploited the NUMA topology by pinning MPI ranks to specific domains using the `ppr:2:numa` mapping. Each node is organized into 4 NUMA groups:

$$\text{NUMA}_i = \{\text{node}_{idx} \mid idx \pmod 4 = i\} \tag{1}$$

By aligning MPI ranks with these physical boundaries, I minimized data movement across the hardware.

### 3.2 Implementation & Optimizations

#### *3.2.1 Domain Decomposition*

The core strategy relies on a two-level domain decomposition. At the top level, MPI divides the global layer into equal parts and assigns them to different ranks. At a deeper level, OpenMP divides each rank's section into smaller chunks that individual threads process independently.

Instead of relying on the standard OpenMP scheduler, I used a custom macro to calculate the indices for each thread. This ensures that every thread's partition starts and ends on a cache line boundary (64 bytes = 16 floats), preventing false sharing between cores.

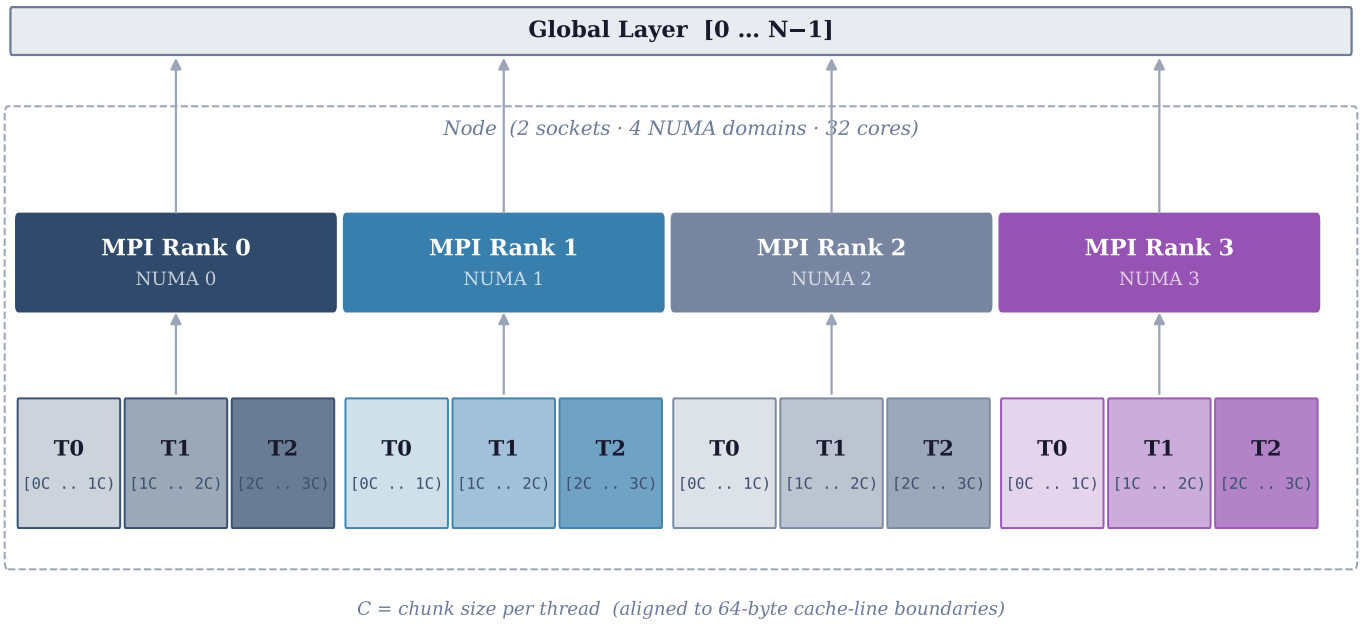


**Figure 1** Hybrid domain decomposition: MPI splits the layer across ranks, then OpenMP further partitions each rank's segment among threads with cache-aligned boundaries.

#### 3.2.2 Memory Management

Manually configuring the memory layout was the most effective way to control cache locality and reduce latency. My strategy focuses on three aspects:

- **Alignment:** All memory buffers are aligned to 64 bytes to match the CPU cache line size, preventing false sharing.
- **First-Touch Policy:** I initialize arrays in parallel using the same static chunking logic used for computation. This ensures the allocation of physical memory on the same socket that will process it.
- **Padding:** Array sizes are rounded to the next multiple of the SIMD width (16 floats) so that vectorized operations never access invalid memory.

#### 3.2.3 Computational Optimizations

To speed up the inner particle loop (the bottleneck), I applied several optimizations while preserving numerical correctness.

- **Distance Look-Up Table:** The frequent square root calculations are replaced by a pre-computed table. Since the distances are discrete integers, fetching a value from memory is faster than computing it.
- **Pre-calculated Constants:** In the energy formula, the division by layer size (repeated millions of times) is replaced by a single pre-calculated constant.
- **SIMD Vectorization:** Using `#pragma omp simd`, the compiler generates AVX2 instructions that process 8 floats simultaneously in 256-bit registers.

#### 3.2.4 Stencil & Communication

To avoid slow array copying during relaxation, I use a two-buffers rotation to swap pointers instead of moving data. For MPI, boundaries are exchanged using non-blocking operations (`MPI_Isend` / `MPI_Irecv`), allowing the CPU to work on interior cells while border data travels in the background. The global maximum is found using `MPI_MAXLOC` to track both the peak energy value and its position.

### 3.3 MPI+OpenMP Results

For all distributed tests, I used the flag `--map-by ppr:2:numa:PE=2` to distribute MPI ranks across NUMA domains.

**Table 3** Strong scaling results.

| Problem | Config | Cores | Time (s) | $S_p$ | $E_p$ |
|---|---|---|---|---|---|
| 30 K | 1 node | 32 | 0.233 | 5.0 | 0.155 |
| | 2 nodes | 64 | 0.041 | 28.3 | 0.443 |
| | 4 nodes | 128 | 0.022 | 52.4 | 0.410 |
| 120 K | 1 node | 32 | 0.482 | 9.6 | 0.299 |
| | 2 nodes | 64 | 0.107 | 43.3 | 0.677 |
| | 4 nodes | 128 | 0.053 | 86.6 | 0.676 |
| 1 M | 1 node | 32 | 0.565 | 12.5 | 0.391 |
| | 2 nodes | 64 | 0.121 | 58.5 | 0.913 |
| | 4 nodes | 128 | 0.070 | 100.5 | 0.785 |

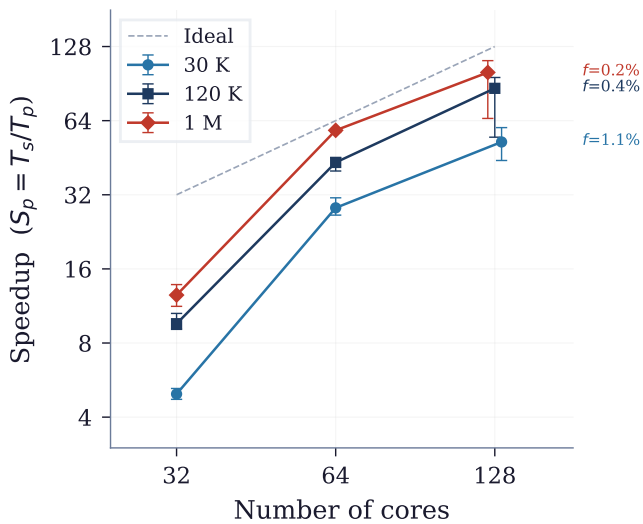


**Figure 2** MPI+OpenMP strong scaling: speedup vs. number of cores for three problem sizes. The line shows ideal linear scaling.

#### 3.3.1 Strong Scaling

In strong scaling, the problem size stays fixed while I increase the number of processes. I define speedup as $S_p = T_s/T_p$ and efficiency as $E_p = S_p/p$.

The single-node configuration is significantly slower than expected. On 32 cores, the 1 M test achieves only 12.5× speedup instead of the theoretical 32×. This is caused by thread overhead: the configuration uses 16 MPI ranks with 4 OpenMP threads each, producing 64 software threads on 32 physical cores. The excess threads compete for execution resources, causing frequent context switching and cache pollution.

The problem disappears on multi-node runs. The jump from 1 to 2 nodes is much larger than 2×: on 30 K, the 2-node configuration is 5.7× faster than 1-node. This reflects the elimination of the exceeding overhead, not just the addition of hardware.

**Table 4** Weak scaling results.

| Prog. | Config | Size | Time (s) | $E_w$ |
|---|---|---|---|---|
| 1 | 1 node | 30 K | 0.233 | 1.00 |
| | 2 nodes | 60 K | 0.060 | 3.86 |
| | 4 nodes | 120 K | 0.053 | 4.37 |
| | 1 node | 1 M | 0.565 | 1.00 |
| | 2 nodes | 2 M | 0.251 | 2.25 |
| | 4 nodes | 4 M | 0.327 | 1.73 |

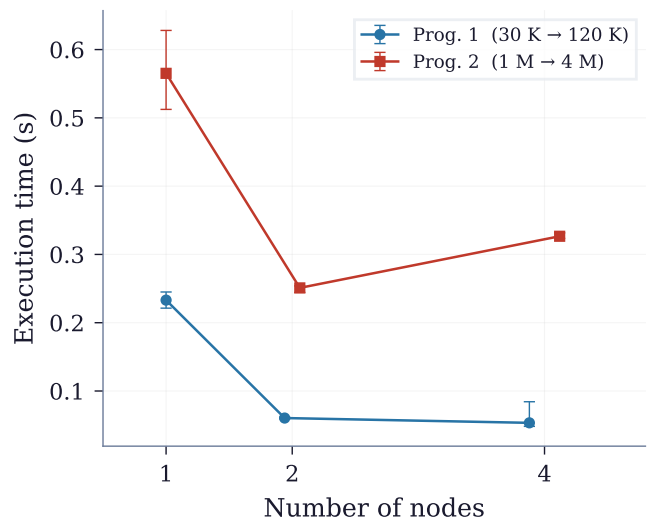


**Figure 3** Weak scaling efficiency across 1, 2, and 4 nodes for both problem progressions.

On 4 nodes (128 cores), the code reaches 100.5× speedup on 1 M and 86.6× on 120 K. Using Amdahl's law, the estimated serial fraction is $f = 0.22\%$ for 1 M, suggesting that the code is highly parallel. The theoretical maximum speedup with infinite cores would be approximately 428×.

The 30 K test scales less efficiently (52.4× on 128 cores, $f = 1.1\%$) because the problem is too small relative to the communication overhead.

**Table 5** Hardware counters from `perf stat` on the 1 M test. Average of 4 runs.

| Metric | Baseline | Parallel |
|---|---|---|
| IPC | 1.21 | 1.32 |
| L1 miss rate | 13.64% | 12.10% |
| Cache miss rate | 1.53% | 4.06% |
| Total instructions | 24.8 G | 39.0 G |
| Total cycles | 20.4 G | 29.5 G |
| Task-clock | 7.7 s | 11.2 s |
| Wall time | 7.08 s | 0.565 s |

#### 3.3.2 Weak Scaling

In weak scaling, I increase the problem size proportionally with the number of nodes. I define weak scaling efficiency as $E_w = T_1/T_p$. Ideally, $E_w = 1$.

Both progressions show super-linear weak efficiency ($E_w > 1$). This is largely an artifact of the 1-node overcommitment: since $T_1$ is inflated, all efficiency values relative to it appear inflated.

Progression 2 shows efficiency decrease at 4 nodes ($E_w$ drops from 2.25 to 1.73). At 4 nodes with a 4 M layer, the sqrt lookup table grows to 15.6 MB, which no longer fits in the L3 cache slice of a single CCX (8 MB on Naples). Since the table is accessed with a random pattern depending on particle positions, this causes cache thrashing. Additionally, the cost of MPI communication grows with the communicator size.

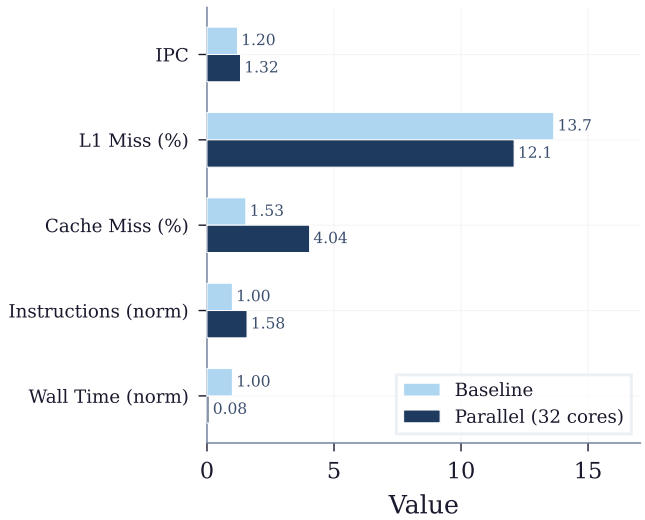


**Figure 4** Hardware counter comparison between the baseline (1 process) and the parallel version (1 node, 32 cores) on the 1 M test.

### 3.4 CPU Profiling Analysis

To understand the performance beyond raw timing, I profiled the 1 M test using `perf stat` in two configurations: the baseline (1 process) and the parallel version (1 node, 32 cores).

The IPC improves from 1.21 to 1.32 because each rank works on a smaller portion of the layer. With 16 MPI ranks on a 1 M layer, each rank processes 62 500 floats. The corresponding layer partition (244 KB) plus the auxiliary array (244 KB) fit entirely in the L2 cache (512 KB per core on Naples), reducing memory stalls.

The L1 miss rate drops from 13.64% to 12.10%, confirming better data locality per core. However, the overall cache miss rate increases from 1.53% to 4.06% because `perf stat` measures the entire process tree, including MPI initialization and halo exchanges, that in any case generate cache-cold accesses.

The total instruction count grows from 24.8 G to 39.0 G, reflecting the overhead of the parallel runtime: OpenMP scheduling, MPI communication, and redundant sqrt table initialization across ranks. The complexive CPU time is 11.2 s (vs. 7.7 s baseline), meaning the parallel version uses 1.5× more total CPU work to achieve a 12.5× improvement.

## 4 GPU ACCELERATION (CUDA)

The second implementation targets the GPU architecture using the NVIDIA CUDA framework.

### 4.1 Device Architecture

I had access to the Quadro RTX 6000 GPU, based on the Turing architecture:

- **CUDA cores:** 4608 (72 SMs, 64 cores/SM).
- **Memory:** 24 GB GDDR6, 384 GB/s bandwidth.
- **L2 cache:** 6 MB.
- **Shared memory:** up to 64 KB per SM.
- **Peak SP performance:** ∼16.3 TFLOP/s.

Threads are grouped into warps of 32, executing the same instruction simultaneously (SIMT model). This is optimal fit for the simulation, where the same energy formula is applied to many data points.

### 4.2 Implementation & Optimizations

#### 4.2.1 Domain Decomposition

Instead of splitting the array into contiguous chunks, I assigned cells using a cyclic pattern:

$$\forall i, \quad \text{thread}_i = \{\text{cell}_j \mid j = i + k \cdot N_{\text{threads}},\ j < S_{\text{layer}}\} \quad (2)$$

This ensures adjacent threads access adjacent memory addresses, allowing the GPU to coalesce multiple reads into single memory transactions.

#### 4.2.2 Memory Management

The most effective optimization was minimizing access to slow global memory by using shared memory. During the stencil phase, threads load a block of the surface into shared memory so that neighbor values can be accessed instantly. For the bombardment phase, I implemented a tiling strategy where threads cooperatively load small chunks of particle data into shared memory.

**Table 6** CUDA performance across problem sizes.

| Size | $T_s$ (s) | CUDA (s) | Speedup |
|---|---|---|---|
| 30 K | 1.157 | 0.021 | 54.9× |
| 60 K | 2.293 | 0.034 | 68.1× |
| 120 K | 4.621 | 0.062 | 74.3× |
| 1 M | 7.080 | 0.071 | 99.9× |
| 2 M | 26.499 | 0.108 | 245.7× |
| 4 M | 58.080 | 0.211 | 274.6× |

#### 4.2.3 Computational Optimizations

- **Pre-calculated energy scaling:** Instead of dividing energy by the layer size inside the inner loop, I perform this while loading particles into shared memory.
- **Hardware square root:** I use the `__fsqrt_rn()` intrinsic to compute square roots directly on the Special Function Units (SFUs). On Turing, each SM has 16 SFUs that can serve a full warp in 2 cycles.
- **Read-only cache:** The `__ldg()` intrinsic forces particle data through the read-only cache.
- **Loop unrolling:** `#pragma unroll` reduces loop overhead in the inner kernels.

#### 4.2.4 Stencil & Synchronization

To avoid array copying during relaxation, I use pointer swapping between two buffers. For the global maximum, I pack both the energy value and its position index into a single 64-bit integer and use `atomicMax` to find the peak in one hardware-level operation.

### 4.3 CUDA Results

Since there is only one GPU, I measure how the speedup over the baseline changes as the problem size grows.

The speedup grows from 54.9× on 30 K to 274.6× on 4 M. The growth accelerates between 1 M and 2 M, where the speedup jumps from 100× to 246×, because the CPU and GPU scale differently.

On the CPU, the 2 M working set (24 MB across three arrays) exceeds the L3 cache capacity, forcing frequent accesses to slow main memory. On the GPU, the 384 GB/s bandwidth handles the growth gracefully: the CUDA time increases only 1.5× from 1 M to 2 M, while the baseline increases 3.7×.

At the small end (30 K), the GPU is underutilized: only ∼59 thread blocks across 72 SMs. At 1 M, ∼1953 blocks fully occupy all SMs.

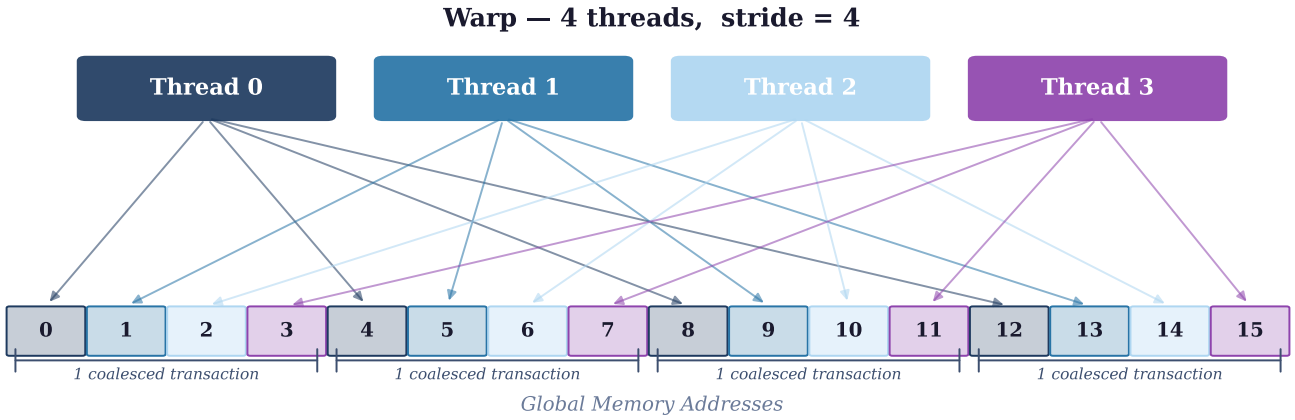


**Figure 5** Coalesced memory access pattern: adjacent threads in a warp access contiguous addresses, enabling efficient memory transactions.

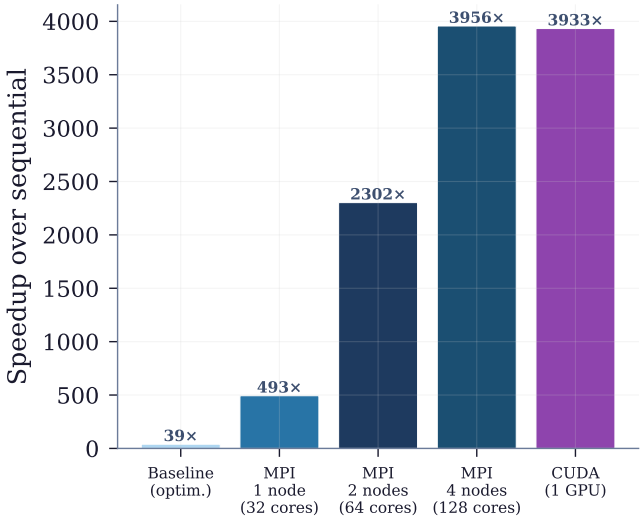


**Figure 6** CUDA speedup over baseline across problem sizes. The dashed line marks 100×.

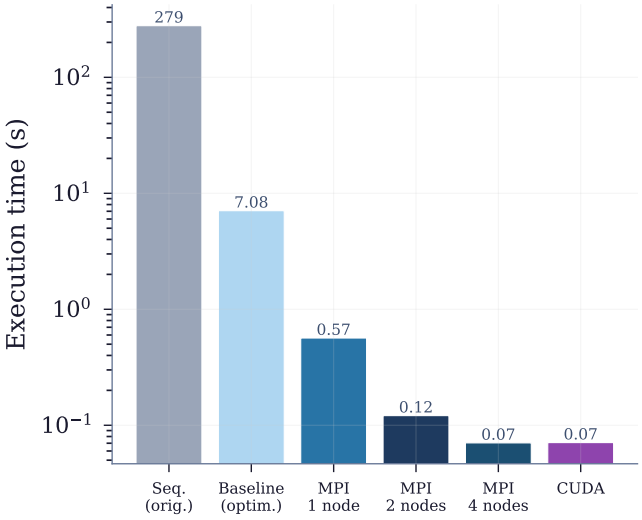


**Figure 7** Absolute execution time: baseline vs. CUDA across problem sizes (log scale).

#### 4.4 GPU Profiling Analysis

I profiled the CUDA execution using Nsight Systems (`nsys`) on the 30 K and 1 M tests.

The `storm_kernel` dominates completely, accounting for over 99% of GPU time. The stencil and reduction kernels are minimals because they perform simple operations on contiguous memory with perfect coalesced access.

Memory transfers are also negligible: H2D takes 0.146 ms on 30 K (0.8%) and 0.015 ms on 1 M (0.02%). The particle data is small (∼160 KB per storm for 20 K particles on the 30 K test, ∼40 KB for 5 K particles on the 1 M test), the layer is initialized on the GPU via `cudaMemset`, and only the maximum value and position come back to the host. The implementation is almost purely compute-bound.

The `storm_kernel` takes 3.207 ms per wave on 30 K and 17.188 ms per wave on 1 M, which is a 5.4× increase for 33× more cells but 4× fewer particles per storm. This sub-linear scaling confirms that 30 K underutilizes the GPU, while 1 M reaches higher efficiency.

**Table 7** CUDA kernel breakdown from `nsys`.

| Test | Kernel | Time | % GPU |
|---|---|---|---|
| 30 K | storm_kernel | 19.243 ms | 99.8% |
| | reduce_max | 0.018 ms | 0.1% |
| | stencil | 0.012 ms | 0.1% |
| | storm_kernel | 68.752 ms | 99.7% |
| | reduce_max | 0.128 ms | 0.2% |
| | stencil | 0.068 ms | 0.1% |

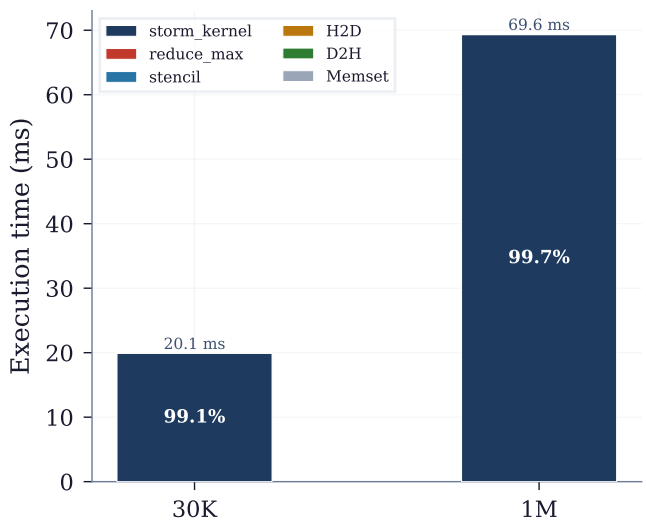


**Figure 8** GPU time breakdown for the 30 K and 1 M tests. The `storm_kernel` accounts for over 99% of execution time in both cases.

### 4.5 Roofline Analysis

To characterize the computational efficiency of the `storm_kernel`, I constructed a roofline model for the Quadro RTX 6000. Since the `ncu` profiler was not accessible due to permission restrictions on the cluster (`ERR_NVGPUCTRPERM`), I derived the arithmetic intensity and throughput analytically from the kernel source code and the `nsys` timing data.

#### 4.5.1 FLOP Estimation

The inner loop of the `storm_kernel` executes the following operations for each (cell, particle) pair: one subtraction and absolute value for the distance, one addition (`dist + 1`), one hardware square root (`__fsqrt_rn`), one hardware division (`__fdiv_rn`), and one accumulation. This amounts to 6 floating-point operations per pair. The total FLOP count for a full run is:

$$\text{FLOP} = S_{\text{layer}} \times N_{\text{particles}} \times 6 \times W \tag{3}$$

where $W$ is the number of storm waves.

**Table 8** Roofline model results for `storm_kernel` (analytical).

| Test | FLOP (G) | Bytes (MB) | AI (F/B) | GF/s | % Peak |
|---|---|---|---|---|---|
| 30 K | 21.6 | 2.4 | ~9 000 | 1 123 | 6.9% |
| 1 M | 120.0 | 32.2 | ~3 750 | 1 745 | 10.7% |

#### 4.5.2 DRAM Traffic Estimation

For each `storm_kernel` invocation, the minimum DRAM traffic consists of one read and one write of the layer array ($S_{\text{layer}} \times 4 \times 2$ bytes), plus one read of the particle position-value array ($N_{\text{particles}} \times 2 \times 4$ bytes, loaded once into shared memory via tiling). The total bytes over all invocations are:

$$\text{Bytes} = W \times \left(S_{\text{layer}} \times 8 + N_{\text{particles}} \times 8\right) \tag{4}$$

This is a lower bound: L2 misses and replay transactions may increase the actual DRAM traffic, but the tiling strategy keeps particle data in shared memory and the layer access pattern is fully coalesced, so the estimate is reasonable.

#### 4.5.3 Results

Both points fall in the compute-bound region of the roofline (Figure 9). With arithmetic intensities of 3 750–9 000 FLOP/byte, the kernel is far above the ridge point ($\approx$ 42 FLOP/byte for DRAM, $\approx$ 8 FLOP/byte for L2), meaning that performance is limited by compute throughput, not memory bandwidth.

The achieved throughput of 1.1–1.7 TFLOP/s represents 6.9–10.7% of the 16.3 TFLOP/s FP32 peak. This gap is expected for a kernel dominated by high-latency special function unit (SFU) operations: each `__fsqrt_rn` and `__fdiv_rn` is dispatched to one of the 16 SFUs per SM, which can serve a full warp in 2 cycles, which is significantly slower than the 64 CUDA cores that can compute a full warp every cycle.

The 1 M test achieves higher throughput than 30 K (1 745 vs. 1 123 GFLOP/s) because the larger grid (1 953 vs. 59 blocks) provides a number of warps to keep all 72 SMs occupied and hide the SFU latency through thread-level parallelism.

## 5 FINAL COMPARISON & CONCLUSION

On the small test (30 K), one GPU and a 128-core cluster achieve nearly identical performance (~21 ms). On 120 K, the MPI cluster is slightly faster (53 ms vs. 62 ms) because domain decomposition allows each rank's chunk to fit

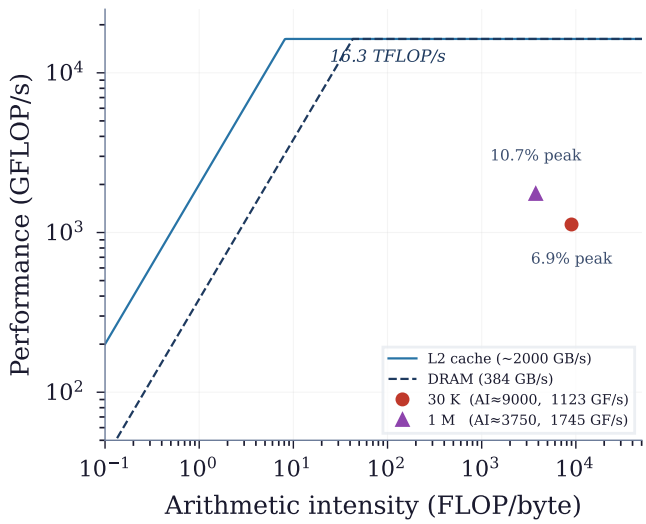


**Figure 9** Roofline model for the Quadro RTX 6000. Both kernel configurations fall in the compute-bound region, reaching 6.9–10.7% of peak FP32 throughput.

**Table 9** Direct comparison on shared problem sizes.

| Size | MPI best | CUDA | MPI $S_p$ | CUDA $S_p$ |
|---|---|---|---|---|
| 30 K | 0.022 s | 0.021 s | 52.4× | 54.9× |
| 120 K | 0.053 s | 0.062 s | 86.6× | 74.3× |
| 1 M | 0.070 s | 0.071 s | 100.5× | 99.9× |
| 2 M | 0.251 s | 0.108 s | 105.6× | 245.7× |
| 4 M | 0.327 s | 0.211 s | 177.3× | 274.6× |

entirely in L2 cache. On 1 M, the two approaches are similar (∼71 ms).

The divergence starts at larger problems. On 2 M, CUDA is 2.3× faster (108 ms vs. 251 ms). On 4 M, CUDA is 1.5× faster (211 ms vs. 327 ms). The CPU baseline decrease to super-linearly above 1 M due to L3 cache overflow, while the GPU's DRAM bandwidth handles the additional data without significant penalty.

Both approaches achieve strong speedups but for different architectural reasons. The MPI+OpenMP version exploits domain decomposition to keep data in fast cache, combined with NUMA placement to minimize memory latency. The CUDA version exploits thread-level parallelism and high memory bandwidth to hide latency.

In practice, if the simulation fits on one GPU, CUDA is the simpler and more cost-effective choice, in these cases one graphics card matches a 128-core cluster. For problems exceeding GPU memory (24 GB on the RTX 6000), MPI across multiple nodes remains the only option.

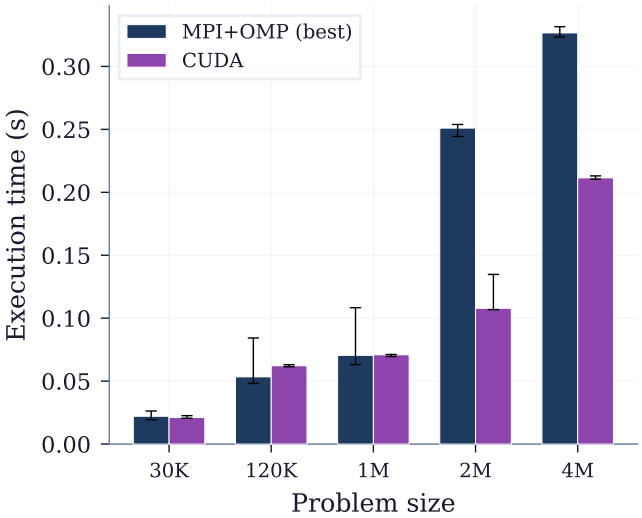


**Figure 10** Execution time comparison between best MPI configuration (128 cores, 4 nodes) and CUDA across problem sizes.